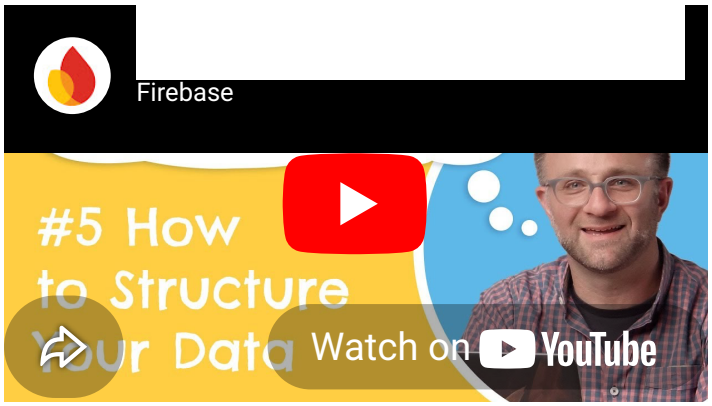
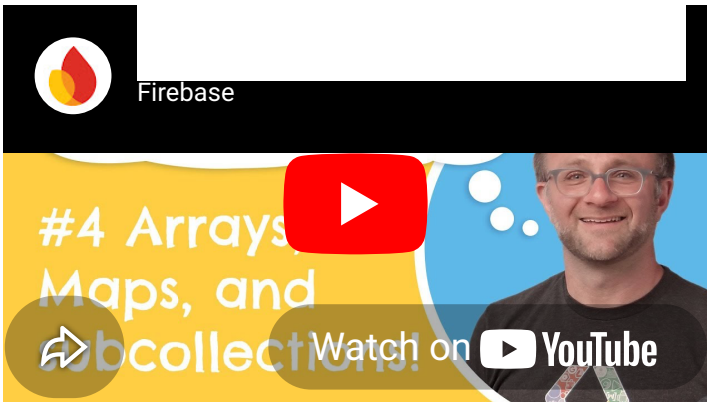
Cloud Firestore Enterprise edition in Native mode is now available! Learn more. (/docs/firestore/enterprise/pipelines-overview)

Choose a data structure





Remember, when you structure your data in Cloud Firestore, you have a few different options:

- Documents
- Multiple collections
- Subcollections within documents

Consider the advantages of each option as they relate to your use case. A few example structures for hierarchical data are outlined in this guide.

Nested data in documents

You can nest complex objects like arrays or maps within documents.

Advantages: If you have simple, fixed lists of data that you want to keep within your documents, this is easy to set up and streamlines your data structure.

Limitations: This isn't as scalable as other options, especially if your data expands over time. With larger or growing lists, the document also grows, which can lead to slower document retrieval times.

What's a possible use case? In a chat app, for example, you might store a user's 3 most recently visited chat rooms as a nested list in their profile.

alovelace

```
name :
  first : "Ada"
  last : "Lovelace"
born : 1815
rooms :
  0 : "Software Chat"
  1 : "Famous Figures"
  2 : "Famous SWEs"
```

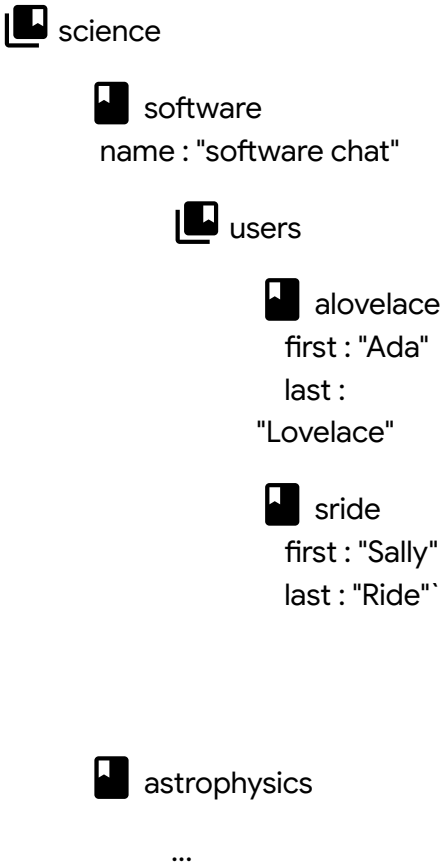
Subcollections

You can create collections within documents when you have data that might expand over time.

Advantages: As your lists grow, the size of the parent document doesn't change. You also get full query capabilities on subcollections, and you can issue collection group queries (https://firebase.google.com/docs/firestore/query-data/queries) across subcollections.

Limitations: You can't easily delete subcollections.

What's a possible use case? In the same chat app, for example, you might create collections of users or messages within chat room documents.



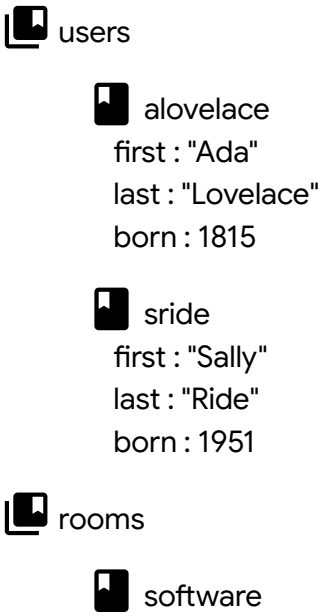
Root-level collections

Create collections at the root level of your database to organize disparate data sets.

Advantages: Root-level collections are good for many-to-many relationships and provide powerful querying within each collection.

Limitations: Getting data that is naturally hierarchical might become increasingly complex as your database grows.

What's a possible use case? In the same chat app, for example, you might create one collection for users and another for rooms and messages.



messages

message1
from : "alovelace"
content : "..."

message2
from : "sride"
content : "..."

Except as otherwise noted, the content of this page is licensed under the Creative Commons Attribution 4.0 License (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the Apache 2.0 License (https://www.apache.org/licenses/LICENSE-2.0). For details, see the Google Developers Site Policies (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-03-27 UTC.